Ayudantía 5: Modelos predictivos de aprendizaje no supervisado
IIC2115

Objetivo del repaso

entender el flujo completo de un problema de aprendizaje no supervisado.

Cosas por abordar:

- ¿Qué diferencia hay entre aprendizaje supervisado y no supervisado?
- ¿Para qué sirven PCA, t-SNE y clustering?
- ¿Por qué es tan importante escalar antes de usar distancias?
- ¿Cómo se elige un número razonable de clústers?
- ¿Cómo interpreto los grupos encontrados?

Motivación desde las dudas frecuentes

En las respuestas del ticket aparecieron varios patrones:

- dificultad para entender **qué se está haciendo realmente**;
- confusión con **PCA**, **clustering** y **K-means**;
- dudas sobre cómo elegir **variables**, **componentes** y **número de clústers**;
- problemas para interpretar gráficos y resultados;
- sensación de que los comandos de `sklearn` son difíciles de encadenar.

Idea

No memorizar comandos, sino construir una forma ordenada de 'pensar' el problema.

Supervisado vs no supervisado

Aprendizaje supervisado

Existe una variable objetivo conocida, es decir, una respuesta que queremos predecir.

- ejemplo: predecir precio, categoría, demanda, riesgo;
- existe un y explícito;
- se evalúa comparando predicción vs valor real.

Aprendizaje no supervisado

No hay una respuesta correcta conocida. Buscamos estructura en los datos.

- ejemplo: encontrar perfiles, patrones, grupos o visualizaciones;
- trabajamos principalmente con una matriz de características X;
- la interpretación requiere más criterio.

Dos familias importantes

En el notebook de clases aparecen dos grandes tareas:

Reducción de dimensionalidad

Transformar muchas variables en pocas dimensiones que resuman parte importante de la información.

- PCA: útil para resumir y visualizar de (patrones más bien lineales)
- t-SNE: útil para visualización, pero debe interpretarse con cuidado (patrones no lineales)

Clustering

Agrupar observaciones similares entre sí, sin conocer etiquetas previamente.

- K-means;
- clustering jerárquico aglomerativo.

Pipeline general del notebook

$$\text{Datos} \rightarrow \text{Preparación} \rightarrow \text{Escalamiento} \rightarrow \text{PCA/t-SNE} \rightarrow \text{Clustering} \rightarrow \text{Interpretación}$$

Idea clave: en no supervisado, el output no viene con una respuesta obvia.

Por eso, después de correr el modelo hay que preguntarse:

- ¿qué patrón apareció?;
- ¿ese patrón tiene sentido para el problema?;
- ¿cómo lo justifico con tablas, métricas o gráficos?;
- ¿sirve para una decisión posterior?

Matriz de características: elegir X

El modelo no supervisado trabaja con variables descriptivas.

En el ejemplo de eficiencia energética:

- compacidad relativa;
- área de superficie;
- área de muros;
- área de techo;
- altura;
- orientación;
- área vidriada;
- distribución del área vidriada.

Ojo

Si se usan variables de desempeño, como carga de calefacción o enfriamiento, hay que tener claro si son parte del clustering o solo se usan después para interpretar.

Ejemplo: construir la matriz X

```
features = [
    "relative_compactness",
    "surface_area",
    "wall_area",
    "roof_area",
    "overall_height",
    "orientation",
    "glazing_area",
    "glazing_area_distribution"
]

X = energy_dataset[features]
```

Lectura:

- `features`: columnas que describen cada observación;
- `X`: matriz que entra al modelo;
- no estamos definiendo todavía un y, porque no es supervisado.

Escalamiento: por qué importa tanto

Muchas técnicas no supervisadas usan **distancias**.

Problema

Si una variable tiene valores mucho más grandes que otra, puede dominar artificialmente el cálculo de distancia.

Ejemplo:

- una columna mide área en cientos de unidades;
- otra columna mide alguna propiedad entre 0 y 1;
- sin escalar, el algoritmo puede darle demasiada importancia al área.

Regla práctica

Antes de PCA, K-means o clustering basado en distancia, normalmente conviene escalar.

Ejemplo: escalar variables

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Interpretación:

- cada variable queda centrada y comparable;
- se reduce el efecto artificial de las unidades de medida;
- el algoritmo trabaja con distancias más razonables.

Duda frecuente

Escalar no cambia el sentido del problema, cambia la escala para que las variables compitan de forma más justa.

PCA: idea central

PCA

PCA construye nuevas variables, llamadas componentes principales, como combinaciones de las variables originales.

¿Para qué sirve?

- resumir información de muchas variables;
- visualizar datos de alta dimensión en 2D o 3D;
- detectar patrones generales;
- reducir ruido o redundancia entre variables.

Idea: las componentes no son columnas originales; son nuevas direcciones que resumen variabilidad.

Ejemplo: PCA en dos componentes

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
```

Lectura:

- `n_components=2`: pedimos dos dimensiones para visualizar;
- `fit_transform`: aprende las componentes y transforma los datos;
- PC1 y PC2: nuevas coordenadas, no variables originales.

Cómo interpretar PCA

PCA se interpreta mirando dos cosas:

1. Varianza explicada

Indica cuánta información, medida como variabilidad, captura cada componente.

- si dos componentes explican poco, el gráfico 2D puede ser limitado;
- si explican mucho, la visualización resume mejor los datos.

2. Cargas o loadings

Muestran cuánto aporta cada variable original a cada componente.

- sirven para darle significado a PC1 y PC2;
- ayudan a evitar interpretar las componentes “a ciegas”.

Varianza explicada y loadings

```
print(pca.explained_variance_ratio_)
print(pca.explained_variance_ratio_.sum())

loadings = pd.DataFrame(
    pca.components_.T,
    columns=["PC1", "PC2"],
    index=features
)
```

Duda frecuente

Si uso dos variables y PCA explica el 100 %, no significa que encontré magia: simplemente no había más dimensiones que resumir.

t-SNE: para visualizar, no para concluir de más

t-SNE

t-SNE busca representar datos complejos en pocas dimensiones, preservando relaciones locales entre observaciones cercanas.

Sirve para:

- explorar visualmente grupos o vecindarios;
- detectar estructuras no lineales;
- complementar PCA cuando la separación no es clara.

Advertencia

Las distancias globales entre grupos en t-SNE no siempre tienen una interpretación directa. Cambios de hiperparámetros pueden cambiar la visualización.

Clustering: qué estamos buscando

Idea central

Clustering busca agrupar observaciones similares, sin usar etiquetas verdaderas.

Lo importante no es solo obtener etiquetas de clúster.

Después hay que interpretar y detenerse a pensar:

- ¿qué caracteriza a cada grupo?;
- ¿qué variables los diferencian?;
- ¿los grupos tienen sentido en el contexto del problema?;
- ¿sirven para explicar o mejorar una decisión posterior?

K-means en simple

K-means necesita que definamos un número de clústers K.

El algoritmo alterna entre:

1. asignar cada observación al centroide más cercano;
2. recalcular centroides como promedio de los puntos asignados;
3. repetir hasta estabilizar.

Ojo

K-means no "descubre" automáticamente el mejor K. Hay que probar valores y justificar una elección razonable.

Probar varios valores de K

```
cluster_var = []

for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        init="k-means++",
        n_init=10,
        random_state=0
    )
    kmeans.fit(X_scaled)
    cluster_var.append(kmeans.inertia_)
```

Idea: entrenar K-means para distintos valores de K y comparar qué tan compactos quedan los grupos.

Método del codo

Inercia

Mide qué tan cerca están los puntos de los centroides de sus respectivos clústers.

Problema:

- al aumentar K, la inercia casi siempre disminuye;
- por eso no basta con elegir el menor valor de inercia;
- buscamos un punto donde seguir aumentando K aporte cada vez menos.

Duda frecuente

El método del codo no entrega siempre una respuesta única. Es una herramienta para justificar, no una receta infalible.

Silhouette score

Idea central

El silhouette score compara qué tan cerca está una observación de su propio clúster versus otros clústers.

Interpretación general:

- valores más altos sugieren grupos más separados y coherentes;
- a diferencia de la inercia, no necesariamente mejora al aumentar K;
- ayuda a comparar configuraciones alternativas.

Ojo

Un buen score no reemplaza la interpretación del problema. Un clúster útil debe tener sentido técnico o práctico.

Elegir K con silhouette

```
silhouette_results = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=0)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_results.append({"k": k, "score": score})
```

Lectura:

- para cada K, entrenamos y medimos separación;
- luego comparamos resultados;
- elegimos una opción razonable y justificable.

Entrenar K-means final

```
best_k = int(silhouette_df.loc[
    silhouette_df["silhouette_score"].idxmax(), "k"
])

kmeans = KMeans(n_clusters=best_k, n_init=10, random_state=0)
kmeans_labels = kmeans.fit_predict(X_scaled)

energy_dataset["kmeans_cluster"] = kmeans_labels
pca_df["kmeans_cluster"] = kmeans_labels
```

Idea clave

El clustering se entrena sobre `X_scaled`. El gráfico en PCA puede usarse solo para visualizar e interpretar.

Advertencia: clustering vs visualización

Confusión típica

Ver los clústers dibujados sobre PC1 y PC2 no significa necesariamente que el modelo se entrenó usando solo PC1 y PC2.

Puede ocurrir que:

- K-means se entrene con las 8 variables escaladas;
- luego se proyecten los resultados en 2D con PCA;
- el gráfico sirva para visualizar, no para definir todo el modelo.

Pregunta guía

Preguntarse, ¿con qué matriz entrené el algoritmo y qué estoy usando sólo para graficar?

Interpretar perfiles de clúster

Una vez obtenidos los clústers, falta aún.

Una buena interpretación suele usar:

- promedios por clúster;
- comparación de variables relevantes;
- gráficos de barras o boxplots;
- una descripción verbal de cada perfil.

Ejemplo de perfil

"Clúster 0: edificios más compactos, menor área de techo y menor carga promedio de calefacción."

Tabla resumen por clúster

```
cluster_summary = energy_dataset.groupby("kmeans_cluster")[
    features + ["heating_load", "cooling_load"]
].mean()

cluster_summary
```

Esto permite responder:

- ¿qué distingue a cada clúster?;
- ¿hay diferencias en desempeño energético?;
- ¿el clustering encontró perfiles útiles o solo grupos difíciles de explicar?

Clustering jerárquico

Idea central

El clustering jerárquico aglomerativo parte con muchas agrupaciones pequeñas y las va fusionando progresivamente.

Comparado con K-means:

- puede producir agrupaciones distintas;
- no usa centroides de la misma forma;
- depende de cómo se mida la cercanía entre grupos;
- sirve como enfoque alternativo para comparar resultados.

Idea clave

Distintos algoritmos pueden definir "grupo" de maneras distintas. Por eso sus resultados no tienen por qué coincidir.

Variables derivadas: cómo pensarlas

Varias dudas del ticket apuntaban a cómo crear variables nuevas.

Una variable derivada debería:

- resumir información relevante;
- tener interpretación clara;
- conectar con el problema;
- no repetir exactamente lo que ya existe;
- poder justificarse en una o dos frases.

Pregunta práctica

Si agrego esta variable, ¿qué nueva lectura del fenómeno me permite hacer?

Variables categóricas

Muchos modelos necesitan datos numéricos.

Si aparece una variable categórica, por ejemplo ubicación o tipo:

- no siempre basta con dejarla como texto;
- a veces hay que codificarla;
- `pd.get_dummies()` puede crear variables binarias;
- algunos resultados aparecen como `True/False`, que pueden convertirse a enteros si es necesario.

Ojo

La codificación debe tener sentido para el modelo. No toda categoría tiene orden numérico natural.

Combinar no supervisado con supervisado

Una duda frecuente fue cómo se conecta esto con modelos predictivos.

Formas típicas de conexión:

- usar clústers como nuevas variables explicativas;
- usar PCA para reducir dimensionalidad antes de entrenar;
- crear perfiles y luego comparar desempeño predictivo;
- segmentar observaciones para entender patrones antes de modelar.

Advertencia

Que un clúster se pueda crear no significa que mejore el modelo predictivo. Hay que evaluarlo.

Checklist para resolver una misión

Antes de escribir código, conviene ordenar el camino:

1. ¿Cuál es el objetivo de la misión?
2. ¿Qué columnas usaré como matriz X?
3. ¿Necesito limpiar, transformar o escalar?
4. ¿Estoy reduciendo dimensión, agrupando o ambas?
5. ¿Cómo elegiré hiperparámetros como K o `n_components`?
6. ¿Cómo interpretaré el resultado?
7. ¿Qué tabla o gráfico justificará mi conclusión?

Dudas frecuentes

1. ¿PCA y clustering se usan juntos?

Sí, pueden usarse juntos, pero hay que distinguir si PCA se usa para visualizar, para reducir dimensión antes de clusterizar, o solo para interpretar.

2. ¿Más variables siempre es mejor?

No. Más variables pueden agregar información, pero también ruido, redundancia o escalas problemáticas.

Dudas frecuentes

3. ¿Cómo justifico el número de clústers?

Puedes usar método del codo, silhouette score, visualización e interpretación del problema. Lo ideal es combinar evidencia cuantitativa y sentido práctico.

4. ¿Qué hago si los clústers no se ven claros?

Revisar variables, escalamiento, outliers, método usado y cantidad de clústers. También puede ocurrir que los datos no tengan grupos naturalmente separados.

Dudas frecuentes

5. ¿Cómo sé qué técnica usar?

Depende del objetivo: si quiero resumir/visualizar, PCA o t-SNE; si quiero encontrar perfiles, clustering; si quiero predecir una variable objetivo, supervisado.

6. ¿Qué significa "interpretar resultados"?

No basta con mostrar un gráfico. Hay que explicar qué patrón aparece, qué variables lo sustentan y por qué importa en el contexto del problema.

Errores comunes

- correr K-means sin escalar variables;
- elegir K solo porque "se ve bonito";
- confundir PCA como si fueran variables originales;
- graficar en PCA y asumir que el modelo se entrenó en PCA;
- crear variables derivadas sin interpretación;
- usar resultados sin justificar qué significan;
- mezclar supervisado y no supervisado sin tener claro el objetivo.

Algunas cosas para tener presente de cara a los ejercicios

1. En no supervisado no hay una respuesta correcta conocida.
2. PCA reduce dimensión; clustering agrupa observaciones.
3. Escalar es fundamental cuando importan las distancias.
4. Elegir K requiere evidencia, no solo intuición.
5. Los clústers deben interpretarse con tablas, gráficos y contexto.
6. Justificar decisiones es esencial.

Revisemos los ejercicios del lunes :)